

PREDICTION TASK  What is the type of task? Which entity are predictions made on? What are the possible outcomes to predict? When are outcomes observed? A Time-Series Forecasting (TSF) task, specifically an Autoregressive (AR) multi-step probability prediction task. The entities on which predictions are made on represent data keys (i.e., unique identifiers for data items). For each key, its access probability over a future time horizon is made (within [0,1]). The outcomes are observed at the end of the predicted time horizon.	DECISIONS  How are predictions turned into actionable recommendations or decisions for the end-user? (Mention parameters of the process / application for this.) The predictions (over a predefined time horizon) are combined with the Confidence Intervals (CIs) (calculated w.r.t. a given confidence level) into a single, composite eviction score for each key (within [0,1]). Specifically, these scores are obtained through a simple formula which applies a weighted sum of usage probabilities and CI. These scores are used to decide which key(s) to remove from the cache when it is full. Specifically, the key(s) with the lowest score(s) in the cache, not belonging to a set of 'protected keys' (which are excluded from eviction), is/are evicted.	VALUE PROPOSITION  Who is the end beneficiary, and what specific pain points are addressed? How will the ML solution integrate with their workflow, and through which user interfaces? The primary beneficiaries are the operations/platform teams integrating the solution into a specific system (direct) and its corresponding end-users (indirect). The key pain points addressed are: 1) Ineffective cache eviction strategies - Rigid, rule-based policies - Poor suitability for real-world, non-stationary data access patterns 2) Inefficient resource utilization - Low cache hit rate - Suboptimal use of fast cache memory 3) High operational and infrastructure costs - Increased load on backing services - Higher infrastructure and operational expenses 4) Degraded service performance and User Experience (UX): - Increased end-user latency - Lower quality of service - Negative impact on UX - Risk of violating Service Level Objectives (SLOs) The solution is integrated via an internal RESTful API. The cache management layer calls the API with recent data accesses information and receives the key(s) to evict whenever a new data item must be inserted into a full cache. The eviction decision is programmatic; no direct user interface is required.	DATA COLLECTION  How is the initial set of entities and outcomes sourced (e.g., database extracts, API pulls, manual labeling)? What strategies are in place to update data continuously while controlling cost and maintaining freshness?	DATA SOURCES  Where can we get data on entities and observed outcomes? (Mention internal and external database tables or API methods.)
IMPACT SIMULATION  What are the cost/gain values for (in)correct decisions? Which data is used to simulate pre-deployment impact? What are the criteria for deployment? Are there fairness constraints?	MAKING PREDICTIONS  Are predictions made in batch or in real time? How frequently? How much time is available for this (including featurization and decisions)? Which computational resources are used? Predictions are made in real-time by the API. The frequency is event-driven, corresponding to how often the cache requires an eviction decision (can range from highly frequent to infrequent). The API must be low-latency; its target average latency should be under 10 milliseconds. The API, organized via a microservice architecture, leverages specialized CPU cores (optimized via quantization) to ensure fast tensor computations.		BUILDING MODELS  How many models are needed in production? When should they be updated? How much time is available for this (including featurization and analysis)? Which computation resources are used?	FEATURES  What representations are used for entities at prediction time? What aggregations or transformations are applied to raw data sources? The keys are represented by three types of input features: - Temporal features (sine time, cosine time): capture the cyclical pattern of the time of day. - Local features (local frequency, local recency): model short-term sequential dynamics of key usage. - Identity feature (request): the raw key is mapped to a low-dimensional, dense vector representation via a learnable embedding layer within the model architecture. The raw data, composed of timestamps (hours of the day) and requests (accessed keys) undergoes a preprocessing pipeline: 1) Missing values removal: raws having any missing value are removed. 2) Trigonometric time encoding: the raw timestamps are transformed into sine and cosine components for cyclical representation. 3) Local frequency: the normalized count of a key's occurrences within the preceding sequence length is calculated. 4) Local recency: the normalized and reversed distance from a key's last occurrence within the lookback window is calculated.
	MONITORING  Which metrics and KPIs are used to track the ML solution's impact once deployed, both for end-users and for the business? How often should they be reviewed?			